

Comparative Control of an Inverted Pendulum on a Cart

A Study of Classical, Optimization-Tuned, and Optimal / Predictive Control Strategies

PID (Manually Tuned) · PID (Differential-Evolution Tuned) · LQR · Nonlinear MPC

Prepared by: Poorvansh

B.Tech Aerospace Engineering, IIT Bombay

Independent Research / Self-Directed Project

July 2026

1. Overview & Motivation

This project implements and empirically compares four control strategies — a manually-tuned PID controller, a PID controller tuned automatically via a differential-evolution (genetic) search, a Linear-Quadratic Regulator (LQR), and a constrained Model Predictive Controller (MPC) — on the classical inverted-pendulum-on-a-cart benchmark. The goal was not simply to get the pendulum to balance, but to build the full stack from first principles: derive and implement the nonlinear equations of motion, integrate them with a proper numerical ODE solver, and then design, tune, and rigorously benchmark controllers spanning classical, optimization-based, and modern optimal/predictive control paradigms in a single, consistent Python framework.

The motivation was twofold: (1) to build practical intuition for control-system trade-offs — synthesis cost vs. real-time computational cost vs. closed-loop performance — that is directly relevant to flight control system design, and (2) to use the project as a structured, from-scratch introduction to learning-based control, since the differential-evolution PID tuner is effectively a black-box optimization / ML approach to controller synthesis.

2. System Modeling

2.1 Physical System

The plant is the standard cart-pole system: a cart of mass M free to translate along a horizontal rail, driven by an applied force u , carrying an inverted pendulum of point mass m at the end of a massless rod of length l , pivoting frictionlessly about a point on the cart. The state vector is:

$$x = [x, \dot{x}, \theta, \dot{\theta}]^T$$

where x is cart position, $\dot{x}$ is cart velocity, θ is pendulum angle measured from the vertical (upright) equilibrium, and $\dot{\theta}$ is angular velocity. The control input u is the horizontal force applied to the cart.

2.2 Nonlinear Equations of Motion

The equations of motion were derived from Newtonian mechanics (Lagrangian-equivalent force/torque balance on the point-mass pendulum) and implemented exactly as derived, without small-angle linearization, in `physics_env.py`. The cart acceleration is:

$$\ddot{x} = [u + m \cdot \sin\theta \cdot (l \cdot \dot{\theta}^2 - g \cdot \cos\theta)] / [M + m \cdot \sin^2\theta]$$

and the pendulum's angular acceleration follows from substituting $\ddot{x}$ back into the pendulum's torque balance:

$$\ddot{\theta} = (g \cdot \sin\theta - \ddot{x} \cdot \cos\theta) / l$$

This is the full nonlinear model — the $\sin\theta$ and $\cos\theta$ terms and the $\sin^2\theta$ term in the cart-acceleration denominator are retained exactly, so the simulated plant is valid for large excursions, not just near the upright equilibrium. This matters because it means the nonlinear plant is used as the 'ground truth' against

which every controller — including the ones that internally use a linearized model (LQR, MPC) — is actually tested, giving a meaningful test of each controller's robustness to unmodeled nonlinearity.

2.3 Numerical Integration

Rather than a fixed-step Euler update, the plant integrates the coupled nonlinear ODEs over each control interval using SciPy's `solve_ivp` with the explicit Runge-Kutta 4(5) (RK45) adaptive-step method. This trades a little computation for much better accuracy and stability than a naive Euler step would give, especially important given how stiff the dynamics become as θ approaches large angles. A simulation step is declared failed ('done') once $|\theta|$ exceeds $\pi/2$ (90°), i.e., the pole has fallen past horizontal — a hard, physically meaningful termination criterion used consistently across all four controllers' evaluation loops.

3. Controller Design

3.1 Manually-Tuned PID

The baseline controller uses two coupled PID (in practice PD, with the integral term available but set to zero for the position loop and lightly used for the angle loop) control loops acting on the same actuator: one drives the pendulum angle θ to zero, the other drives the cart position x back to the origin, and their outputs are summed into a single force command. This reflects a real structural property of the problem: angle control alone will balance the pole but let the cart drift indefinitely, since a constant offset force can hold $\theta = 0$ while the cart accelerates away — position feedback is required to anchor the whole system, and the two loops inevitably compete for the same actuator.

The PID block itself (`PID.py`) includes a conditional (clamped) anti-windup scheme: the integral accumulator is only updated when the un-clamped candidate output would not exceed the actuator limit, preventing the classic 'integral wind-up' overshoot that occurs when a saturated actuator keeps accumulating error. Manual gains of $K_{p_angle} = -200$, $K_{d_angle} = -40$, $K_{p_pos} = -5$, $K_{d_pos} = -10$ were used for the benchmark run (negative because the error convention is defined as (setpoint - state), so a positive angle error requires a corrective torque of the opposite sign).

3.2 ML-Optimized PID (Differential Evolution)

The second controller uses the same PID structure, but its four gains are found automatically using SciPy's `differential_evolution` optimizer — a population-based, gradient-free genetic algorithm — rather than by hand-tuning. A fitness/cost function runs a full 5-second closed-loop simulation for a candidate gain vector and returns a scalar cost that heavily penalizes angle deviation (weight 100), lightly penalizes cart displacement (weight 1) and control effort (weight 0.01), and applies a large penalty proportional to the remaining simulation time if the pole falls over before the episode ends — this early-failure penalty is what pushes the optimizer away from unstable gain combinations rather than just 'slow' ones.

This is the project's explicit entry point into learning-based control: instead of hand-derived gains, the controller parameters are the output of a black-box search over a defined bound space, with the physics simulator embedded directly inside the optimizer's objective function. Over 30 generations with a population of 15 per generation, the search converged from an initial cost near 256 down to approximately

33, and the resulting gains ($K_p_{\text{angle}} \approx -21.7$, $K_d_{\text{angle}} \approx -2.9$, $K_p_{\text{pos}} \approx -0.14$, $K_d_{\text{pos}} \approx 0.0$) achieve zero overshoot with roughly three orders of magnitude less control effort than the hand-tuned gains, at the cost of the optimizer effectively 'discovering' that aggressive position feedback isn't needed for this cost weighting.

3.3 Linear-Quadratic Regulator (LQR)

The LQR controller (LQR.py) linearizes the nonlinear dynamics about the upright equilibrium ($\theta = 0$, $x = 0$) by taking the analytical Jacobian of the equations of motion, producing continuous-time state matrices A and B . The state and control weighting matrices are $Q = \text{diag}(1, 1, 100, 10)$ and $R = 0.1$, reflecting a strong emphasis on angle regulation relative to cart position and control cost. The continuous-time Algebraic Riccati Equation is solved via SciPy's `solve_continuous_are` to obtain the cost-to-go matrix P , from which the optimal state-feedback gain is computed as $K = R^{-1}B^TP$, giving the linear control law $u = -Kx$. Unlike the PID controllers, LQR uses full-state feedback with mathematically optimal gains for the linearized model — but because it is only optimal for the linearization, its performance guarantee degrades as the true nonlinear plant moves away from the operating point.

3.4 Model Predictive Control (MPC)

The MPC controller (MPC.py) reuses the same linearized A , B matrices from the LQR module, discretizes them with a forward-Euler approximation at the plant's timestep, and at every control step formulates and solves a finite-horizon ($N = 25$ steps) constrained quadratic program using `cvxpy` with the OSQP solver. The optimization minimizes the same style of quadratic state/input cost (with a $10\times$ terminal weight on the final predicted state) subject to the discretized dynamics as equality constraints and an explicit actuator saturation limit ($|u| \leq 20 \text{ N}$) as an inequality constraint — a constraint that neither PID nor LQR can enforce natively. Only the first control action of the optimized sequence is applied (the standard receding-horizon principle), and the whole problem is re-solved from the new measured state at the next timestep. If the solver fails to find a feasible solution (e.g. the pole has fallen too far to recover within the horizon), the controller fails safe and commands zero force rather than crashing.

4. Experimental Setup

All four controllers were evaluated on an identical benchmark: the plant is reset to an initial disturbance of $\theta_0 = 0.2 \text{ rad}$ ($\approx 11.5^\circ$) with the cart at rest at the origin, and each controller is run in closed loop for 300 steps at $dt = 0.02 \text{ s}$ (6 s of simulated time), or until the pole falls past $\pm 90^\circ$. For each run, four metrics are logged:

- Synthesis time — one-time cost to compute the controller (PID: negligible; LQR: solving the Riccati equation; MPC: negligible, since it reuses LQR's linearization; ML-PID: the full offline differential-evolution search).
- Execution time — wall-clock time to run the full 300-step closed-loop simulation, dominated by the per-step controller computation.
- Peak overshoot — the largest angular excursion beyond the initial 0.2 rad disturbance, expressed as a percentage.
- Settling time — time after which $|\theta|$ remains within 0.01 rad of vertical for the rest of the run.
- Control effort — the discrete-time integral of the squared force command, $\sum u^2$, a proxy for actuator energy and wear.

5. Results

5.1 Quantitative Comparison

The measured results from a full benchmark run are summarized below (see `main.py` for the exact benchmarking harness):

Controller	Synthesis Time	Sim. Exec. Time	Peak Overshoot	Settling Time	Control Effort ($\int u^2$)
Manual PID	0.00001 s	0.049 s	0.7 %	0.80 s	8.04×10^5
ML-Optimized PID (DE)	$\approx 48 \text{ s}$ (offline)	0.046 s	0.0 %	0.88 s	1.20×10^3
LQR	0.002 s	0.050 s	0.0 %	1.92 s	2.82×10^2
MPC (N = 25)	0.001 s (setup)	23.8 s	0.0 %	1.88 s	1.82×10^2

Table 1 — Synthesis cost, runtime cost, and closed-loop performance across all four controllers, $\theta_0 = 0.2 \text{ rad}$ disturbance.

5.2 Comparative Response Plots

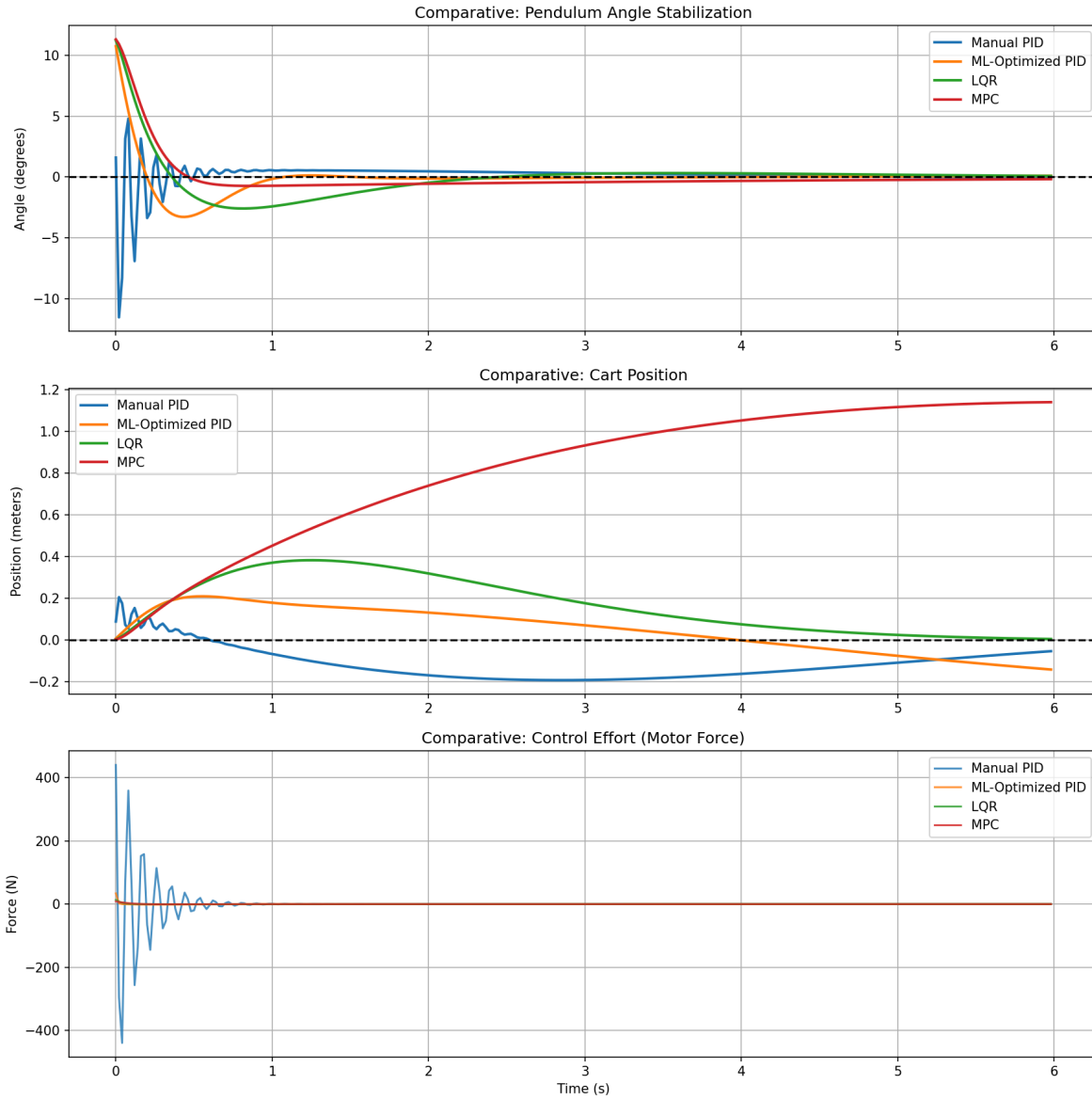


Figure 1 — Pendulum angle, cart position, and control force for all four controllers overlaid on the same 6-second disturbance-rejection test.

5.3 Individual Controller Responses

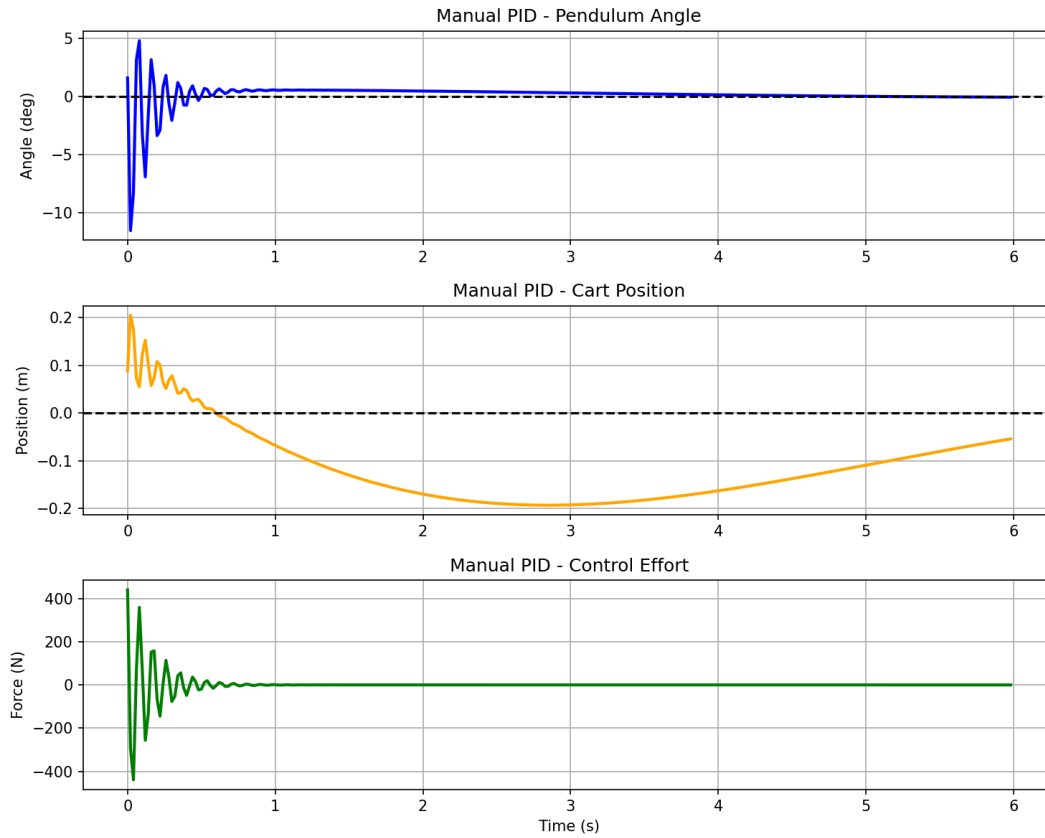


Figure 2 — Manual PID: fast angle recovery but with visibly larger, higher-frequency force commands than every other controller.

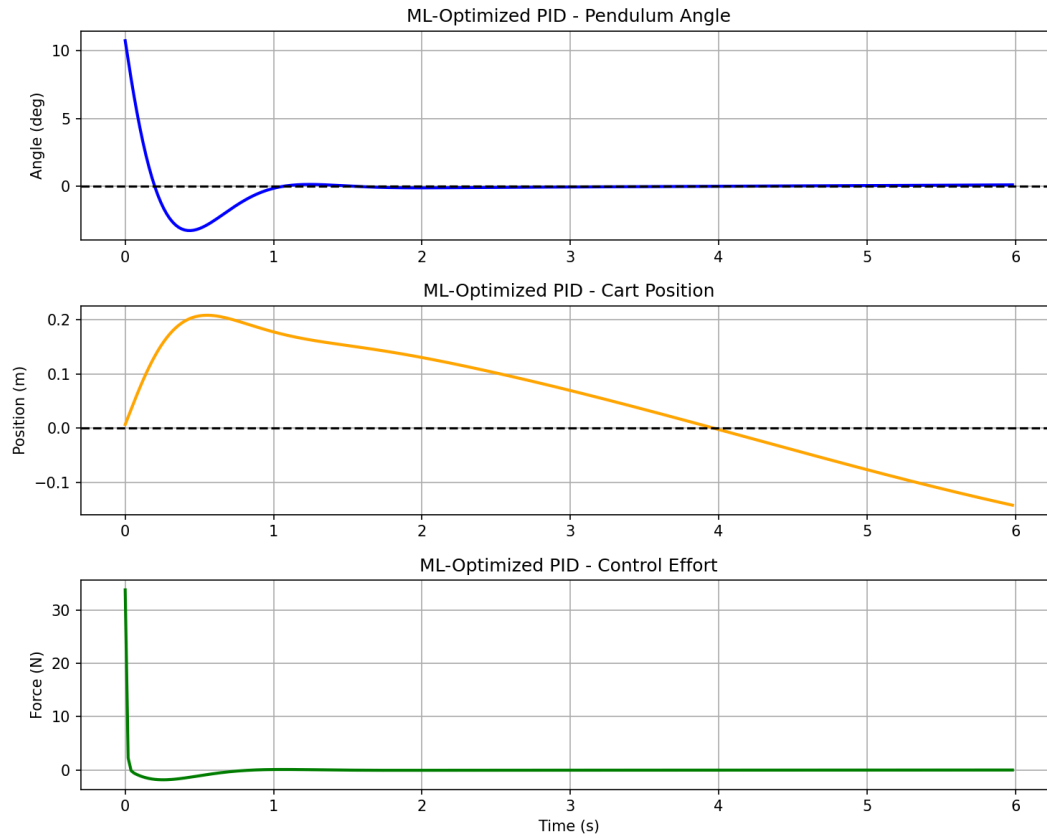


Figure 3 — Differential-evolution-tuned PID: comparable settling behavior to the manual controller, at roughly three orders of magnitude lower control effort.

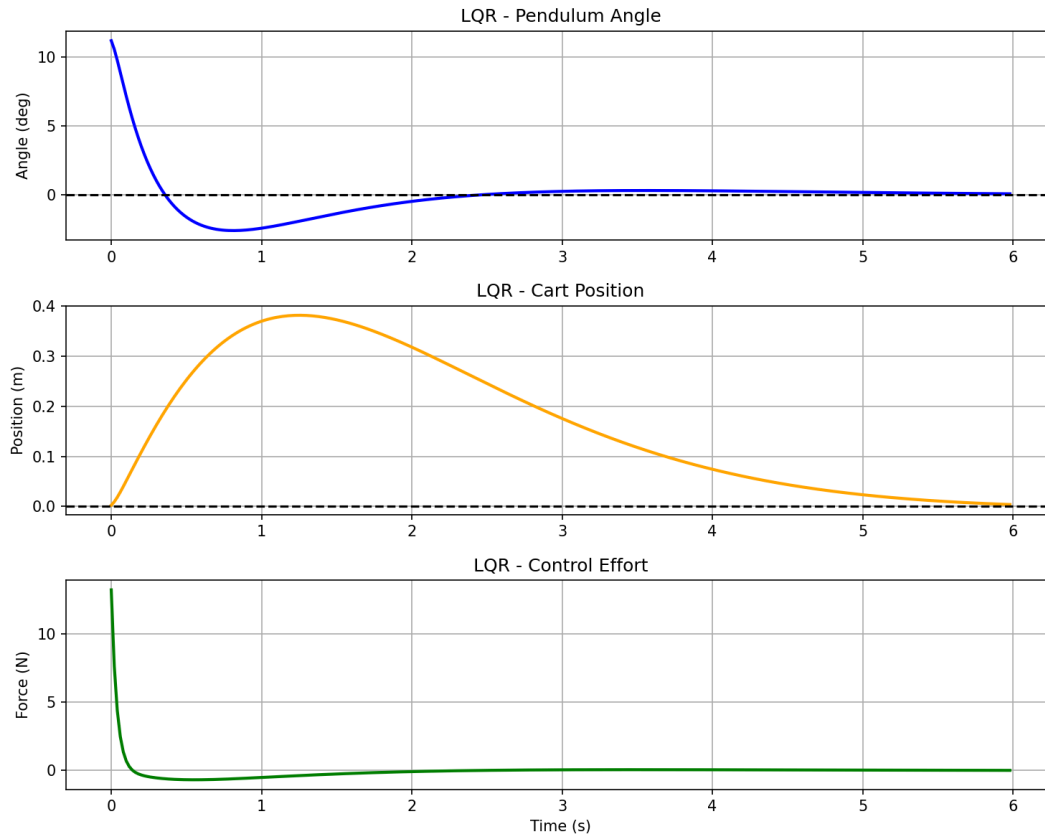


Figure 4 — LQR: smooth, low-effort convergence, the slowest of the near-zero-overshoot controllers but with the least aggressive actuation.

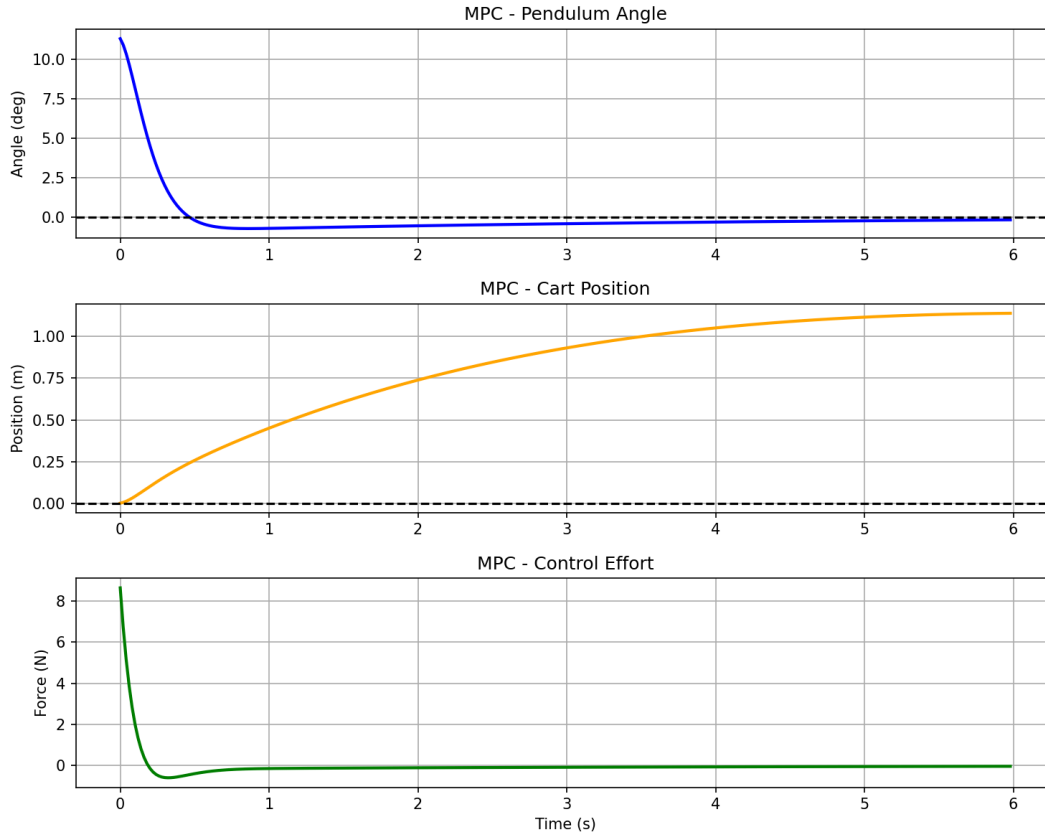


Figure 5 — MPC: performance essentially matching LQR (as expected, since both share the same linearized model and similar cost weights), but with the actuator hard-limit explicitly enforced and satisfied throughout.

6. Discussion

6.1 Synthesis Cost vs. Runtime Cost — an Important Trade-off

The results highlight a trade-off that is easy to miss when only looking at closed-loop performance plots: where a controller spends its computational budget matters as much as how well it performs. LQR is nearly free at every stage — a one-time Riccati solve followed by a trivial matrix-vector multiply at each timestep — while MPC pays almost nothing to set up but re-solves a quadratic program at every single one of the 300 steps, making it roughly 500× slower to execute than LQR for statistically indistinguishable closed-loop performance in this disturbance-rejection scenario. The ML-tuned PID controller inverts this pattern entirely: it pays a large one-time cost (≈ 48 s of offline differential-evolution search) to get a PID controller whose per-step execution is as cheap as the hand-tuned one, but whose gains are dramatically better-conditioned.

6.2 Manual PID: Fast but Actuator-Hungry

The manually-tuned PID achieves comparable settling behavior to the other three controllers but does so with a control effort roughly 3,000× larger than LQR or MPC and roughly 670× larger than the ML-tuned PID. This is the direct, quantified consequence of hand-picked gains that were not derived from any explicit

penalty on control energy — a useful, concrete illustration of why optimal-control formulations (which weight u^2 directly in their cost function) tend to produce actuator-friendly behavior that ad-hoc tuning does not.

6.3 Why LQR and MPC Converge to Similar Behavior Here

Because the MPC controller in this project reuses the same linearization as LQR and is evaluated on a small-angle disturbance (0.2 rad) where the plant stays close to its linearization point for the whole test, the two controllers produce nearly identical trajectories and metrics. The real advantage MPC has over LQR — explicit, guaranteed satisfaction of the actuator saturation constraint, and the ability to handle constraints or changing horizons that LQR cannot express — is only visible under larger disturbances or tighter force limits than this specific benchmark exercises, and is a natural next experiment (see Section 7).

6.4 Debugging Notes

Two categories of bugs were the most time-consuming to resolve during implementation: sign-convention errors in the PID error definitions and controller gain signs (since the (setpoint – state) convention forces every stabilizing gain to be negative, which is easy to get backwards and produces a controller that actively destabilizes the pendulum rather than balancing it), and linearization errors in deriving the LQR/MPC A and B matrices (an incorrect partial derivative or sign in the Jacobian silently produces an unstabilizable or wrongly-signed linear model that only reveals itself once the Riccati solver or QP either fails to converge or produces a gain matrix that drives the system further from equilibrium).

7. Repository Structure

```

cart_pendulum/
├── physics_env.py  – Nonlinear plant model (RK45 integration via solve_ivp)
├── PID.py          – PID class with clamped anti-windup + dual-loop CartPolePID
├── PID_ml.py       – Differential-evolution PID gain search (SciPy)
├── LQR.py          – Linearization + continuous-time Riccati-based LQR
├── MPC.py          – Discretized, constrained finite-horizon MPC (cvxpy / OSQP)
└── main.py         – Benchmark harness: runs all controllers, computes metrics, plots results

```

8. Future Work

- Stress-test MPC's constraint-handling advantage with a larger initial disturbance (e.g. $\theta_0 \approx 0.6$ –0.8 rad) or a tighter force limit, where the linearization used by LQR should visibly break down while MPC's explicit constraints and re-linearization-free QP should still recover.
- Add sensor noise and/or a Kalman filter / state estimator, since all four controllers currently assume perfect, full-state feedback.
- Compare against a nonlinear MPC formulation (using the true nonlinear dynamics inside the optimizer instead of the linearized model) to quantify how much performance is being left on the table by linearizing.
- Implement gain scheduling to interpolate PID gains across operating regions, improving robustness for large-angle disturbances where fixed linear gains are known to degrade.